

COMP 3030 Assignment Part 2 – OpenMP Implementation and Performance

Pahul Preet Singh – Student ID: a1911457 | Repository:
github.com/kangpahul9/pdcassignmentpart2

1. Overview

In this submission, I have parallelized the 2-D N-body problem solver through OpenMP. In part 2A, I have modified the reduced force computation algorithm such that the force computation from all pair interactions takes place in a single force array in a parallel environment through (i) use of a critical section and (ii) OpenMP locks. In part 2B, I measure the effects of force computation algorithm (Basic vs Reduced) and the OpenMP loop schedule on run time, speed-up, efficiency, and scalability.

2. Part 2A Synchronization

Source of the race. The iteration part of the force loop calculates the pairs (part, k), $k > \text{part}$, only once and writes equal and opposite values to `forces[part]` and `forces[k]`. Since iterations are performed by different OpenMP threads, two threads may modify the same array element at the same time (for instance, threads performing calculations for `part = 0` and `part = 1` write to `forces[2..n-1]`). The fact that `+=` consists of a separate load, add and store means that one operation may override another one and the contribution will be lost. Parallel program executed without any synchronizations (4 threads, $n = 100$, 100 steps) resulted in 774, 764 and 784 different lines compared to the reference execution (three consecutive runs): incorrect and non-deterministic output.

Data sharing for default(none). `step` and `part` is private since all OpenMP threads go through the whole timestep loop and should have their own counters; otherwise, the shared `step` will be incremented by all the threads. `curr` (states of particles) and `forces` are shared since all the threads work with one instance of these data. `n`, `n_steps`, `delta_t` and `output_freq` are read-only shared variables and it is safe to share them. In the lock version the `locks` array is shared since all the threads should use one set of locks. Temporary variables like `f_part_k`, `len` and `fact` are automatic variables of `Compute_force` routine and hence are private to each thread.

Stage synchronization. Every phase (apply reset forces, calculate forces, update bodies) is its own `omp for`; the implicit barrier at the end of the force loop ensures that no body is updated until all force calculations are complete, while the barrier at the end of the update loop ensures that positions are consistent for the next stage. Output occurs within `omp single`. The force loop uses `schedule(static,1)`, as iterations involve $n-1$ interactions (triangular work).

Comparison of critical and locks. `part2a_critical.c` uses only the critical construct with the name `(critical(force_update))` to encapsulate all four updates; however, the distance and force calculations remain outside of it. All updates are serialized using only one global mutual exclusion. `part2a_locks.c` defines one `omp_lock_t` per particle (`lock[part]` protects `forces[part]`) and initializes/deinitializes all these locks before and after the parallel region respectively. Threads block only in case if they access the same particle. Calculation of the contribution to `forces[part]` is performed using a private variable and updated once under `lock[part]` to decrease the number of lock acquisitions for it from $n-1-\text{part}$ to one. A thread never owns more than one lock at the same time which eliminates a possibility of a deadlock. In a short test run (2 threads, $n = 2000$, 20 iterations) the critical version took 0.976 s and the lock version 0.351 s.

3. Part 2A Validation

Both the codes were executed with 1, 2, 4, 8 and 16 OpenMP threads with 100 bodies, 100 steps, and printing after every 10 steps and the total output of both codes was compared with the provided sequential code `nbody_shared_forces` except the elapsed-time line: all comparisons showed 0 differences. Ten executions with 8 threads showed matching results, but the unsynchronized one failed all the times. Both codes also generate the same results as the sequential one for the public data with $n=16$ and $n=1$ (using option i) with 1, 2, 4 and 8 threads.

4. Experimental Environment and Methodology

Experiments were conducted on an Apple MacBook Pro with an M3 Pro chip (12 cores: 6 performance and 6 efficiency cores), using Apple clang 15.0.0 and Homebrew libomp, compiling with `-O2` and `-DNO_OUTPUT` such that only elapsed time is outputted. The four implementations had exactly the same parameters set: $n=4000$ randomly generated particles, 100 timesteps, $dt=0.01$. For thread numbers 1, 4, 8, 16 and 32, one run was discarded as warm-up and five runs measured for timing. The programs measure the time for the timestep loop with `omp_get_wtime()`, the average is calculated and returned. The computer was plugged into mains power and no other applications running. Speedup is defined as $S(p) = T(1)/T(p)$ for each implementation relative to itself at one thread, and efficiency is defined as $E(p) = S(p)/p$. The script `scripts/bench.sh` and raw timing results are available in `results/raw_times.csv`.

5. Results

Table 1: Mean execution time in seconds ($n = 4000$, 100 steps, 5 runs)

Threads	Basic	Reduced Default	Reduced Forces Cyclic	Reduced All Cyclic
1	1.984	2.420	2.503	2.559
4	0.533	1.058	0.654	0.664
8	0.459	0.569	0.457	0.462
16	0.340	0.389	0.339	0.355
32	0.338	0.354	0.341	0.356

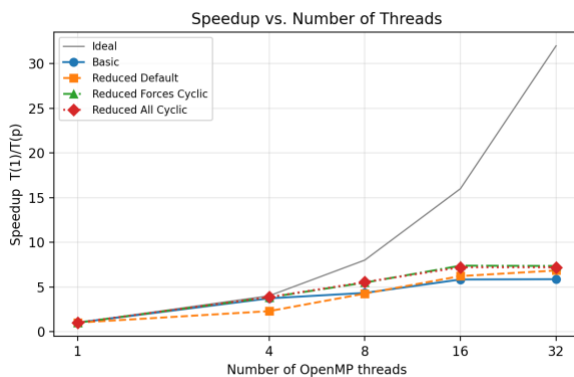


Figure 1: Speedup vs. number of threads

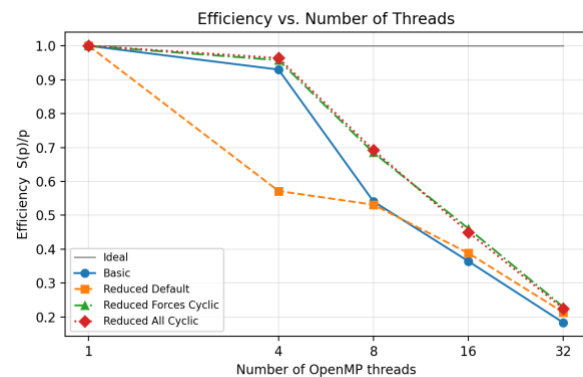


Figure 2: Efficiency vs. number of threads

6. Performance Analysis

Basic vs Reduced. Even though Reduced does only about half as many force calculations, Basic was faster with one thread (1.984 s compared to 2.42–2.56 s). In Basic, every time the iteration adds a force on one particle to an accumulating sum once; it is the same process each time, which makes it easy for the compiler to optimize. The Reduced loop iterates $\text{forces}[k] -= \dots$, where k changes, so there is a memory store operation in every iteration; it also has to zero every iteration a $\text{thread_count} * n \text{ loc_forces}$ array and add the result to forces. In this particular computer, the cost of those memory accesses overrode the savings of the computation.

Scheduling comparison. The default (block) schedule gave poor results for Reduced with 4 threads (1.058 s, $S = 2.29$, $E = 0.57$), since iteration part has $n-1$ -part interaction: the first thread gets the heaviest fourth of iterations, approximately 43.8% of all pairs, which gives a speedup bound of $1/0.438 \approx 2.29$, precisely what was measured. With 8 threads, the first block contains 23.4% of the pairs (bound ≈ 4.27 ; measured 4.25). The cyclic iteration schedule equalizes the heavy and light iterations and gets rid of this imbalance (Forces Cyclic: 0.654 s, $S = 3.83$, $E = 0.96$ with 4 threads). The cyclic iteration schedule for all loops did not help any further, as the zeroing, summation, and update loops have an equal amount of work per iteration, and the cyclic distribution makes sure that the neighboring elements belong to different threads, which made All Cyclic schedule equally or slightly worse than Forces Cyclic (0.355 s compared to 0.339 s with 16 threads). The overhead of the scheduling itself is insignificant for static schedules; the impact comes from the load and memory access.

The speedup, efficiency, and scalability. For up to 4 threads, Basic, Forces Cyclic and All Cyclic are very scalable ($E \geq 0.93$). Then efficiency drops for each variant: 8 threads yield $S = 5.48$ ($E = 0.68$) for Forces Cyclic and just 4.32 ($E = 0.54$) for Basic. M3 Pro has 6 performance and 6 efficiency cores, and static scheduling gives equal work to each thread; threads working on slower efficiency cores end last, and the other threads wait at the barrier. Fixed overheads (barriers, zeroing and summing loc_forces , whose size depends on the number of threads) take up a bigger share of the decreasing share of the work of each thread. 16 threads perform better than 8, since all 12 cores are busy (the fastest is Forces Cyclic with 0.339 s and $S = 7.38$). At 32 threads, 12 cores are oversubscribed, additional threads merely share cores, and execution time is constant (0.338 – 0.356 s), but efficiency is halved (≈ 0.2).

7. Conclusion

The proper synchronization of parallel accumulation of the force array will be performed by means of the lock mechanism; fine grained per-particle locks together with the use of the private partial sum were far less expensive than the lock on the global critical section. As for Part 2B, the scheduling of the force loops was an essential factor for the Reduced algorithm, where the triangular force loop requires the cyclic scheduling strategy, and any application of the same for the uniform force loops does not make sense. Therefore, the Reduced Forces Cyclic proved to be the most effective version of the program in terms of speedup and efficiency at 8 to 16 threads and the fastest time at 16 threads (0.339 sec).